TEE(Trusted Execution Environment) 동작 원리 상세 가이드

목차

1. TEE 기본 개념과 동작 원리
2. Intel TDX 아키텍처
3. 메모리 격리와 보호 메커니즘
4. 증명(Attestation) 시스템
5. 측정(Measurement)과 신뢰 체인
6. 암호화 키 관리
7. 보안 부팅 과정
8. 런타임 보안 메커니즘
9. 실제 동작 시나리오
10. 보안 위협과 대응

1. TEE 기본 개념과 동작 원리

1.1 TEE란 무엇인가?

TEE(Trusted Execution Environment)는 **하드웨어 기반의 보안 격리 환경**입니다. 일반적인 운영체제나 하이퍼바이저로부터도 격리되어 있어, 심지어 시스템 관리자나 루트 권한을 가진 사용자도 TEE 내부의 코드와 데이터에 접근할 수 없습니다.

1.2 TEE의 핵심 원리

```
┌─────────────────────────────────────────────────────────────┐
│                          물리적 하드웨어                      │
├─────────────────────────────────────────────────────────────┤
│                      CPU 하드웨어 보안 기능                   │
│ ┌──────────────────┐ ┌──────────────────┐ ┌───────────────┐  │
│ │     일반 영역     │ │     TEE 영역      │ │     보안 키    │  │
│ │    (Rich OS)     │ │  (Secure World) │ │  (Hardware)  │   │
│ │                  │ │                  │ │               │  │
│ │ - Linux/Windows │ │ - 격리된 실행     │ │ - 암호화 키    │  │
│ │ - 일반 앱         │ │ - 보안 앱         │ │ - 증명 키     │  │
│ │ - 하이퍼바이저     │ │ - 신뢰 체인       │ │ - 루트 키     │  │
│ └──────────────────┘ └──────────────────┘ └───────────────┘  │
└─────────────────────────────────────────────────────────────┘
```

1.3 TEE의 보안 속성

1.3.1 격리(Isolation)

- **메모리 격리**: TEE 메모리는 일반 OS에서 접근 불가
- **실행 격리**: TEE 코드는 독립적인 실행 컨텍스트
- **I/O 격리**: 보안이 중요한 I/O는 TEE를 통해서만 처리

1.3.2 무결성(Integrity)

- **코드 무결성**: 실행되는 코드가 변조되지 않았음을 보장
- **데이터 무결성**: 처리되는 데이터가 변조되지 않았음을 보장
- **실행 무결성**: 예상된 순서대로 코드가 실행됨을 보장

1.3.3 기밀성(Confidentiality)

- **데이터 암호화**: 메모리 내 데이터 자동 암호화
- **통신 암호화**: TEE 간 통신 암호화
- **키 보호**: 암호화 키의 하드웨어 기반 보호

---

2. Intel TDX 아키텍처

2.1 TDX 개요

Intel TDX(Trust Domain Extensions)는 Intel의 TEE 구현으로, **가상화 기반의 격리**를 제공합니다.

```
┌──────────────────────────────────────────────────────────────┐
│                              Host OS                          │
├──────────────────────────────────────────────────────────────┤
│                            Hypervisor                          │
│ ┌──────────────────┐  ┌──────────────────┐  ┌────────────────┐ │
│ │      일반 VM     │  │  Trust Domain    │  │    다른 TD     │ │
│ │                  │  │       (TD)       │  │                │ │
│ │ - 일반 게스트OS  │  │ - 보호된 VM      │  │ - 독립적 TD    │ │
│ │ - 일반 앱        │  │ - 격리된 메모리  │  │ - 별도 측정    │ │
│ └──────────────────┘  └──────────────────┘  └────────────────┘ │
```

```
├──────────────────────────────────────────────────────────────┤
│                        TDX Module (SEAM)                     │
├──────────────────────────────────────────────────────────────┤
│                       Intel CPU Hardware                     │
│ ┌─────────────────┐  ┌─────────────────┐  ┌───────────────┐  │
│ │      Memory     │  │      Crypto     │  │  Attestation  │ │
│ │    Encryption   │  │     Engines     │  │   Hardware    │ │
│ └─────────────────┘  └─────────────────┘  └───────────────┘  │
└──────────────────────────────────────────────────────────────┘
```

2.2 TDX 핵심 컴포넌트

2.2.1 SEAM (Secure Arbitration Mode)

- **역할**: TDX의 핵심 보안 모듈
- **기능**:
- TD(Trust Domain) 생성 및 관리
- 메모리 암호화 키 관리
- 증명 데이터 생성

2.2.2 TD (Trust Domain)

- **정의**: 보호받는 가상 머신
- **특징**:
- 독립적인 메모리 공간
- 고유한 암호화 키

- 별도의 측정값

2.2.3 TDVF (TDX Virtual Firmware)

- **역할**: TD의 부트 펌웨어
- **기능**:
- 보안 부팅 수행
- 초기 측정값 생성
- 게스트 OS 로딩

2.3 TDX 동작 과정

2.3.1 TD 생성 과정

```
1. 하이퍼바이저가 TDX 모듈에 TD 생성 요청
↓
2. SEAM이 새로운 TD를 위한 메모리 영역 할당
↓
3. 메모리 암호화 키 생성 및 할당
↓
4. TD의 초기 측정값(MRTD) 계산
↓
5. TDVF 로딩 및 실행 시작
↓
6. TD 내부에서 게스트 OS 부팅
```

2.3.2 메모리 접근 제어

```
일반 VM의 메모리 접근:

Host OS → Hypervisor → VM Memory (접근 가능)



TD의 메모리 접근:

Host OS → Hypervisor → TD Memory (접근 차단!)

↓

TDX Module → TD Memory (암호화된 접근만 허용)
```

---

3. 메모리 격리와 보호 메커니즘

3.1 메모리 암호화

3.1.1 TME (Total Memory Encryption)

```
물리 메모리 레이아웃:



┌──────────────────┐ ← 물리 주소 공간

|    일반 메모리    | (평문 저장)

├──────────────────┤

|    TD 메모리 #1  | (키 A로 암호화)

├──────────────────┤

|    TD 메모리 #2  | (키 B로 암호화)

├──────────────────┤
```

```
|    TD 메모리 #3  | (키 C로 암호화)
└──────────────────┘
```

3.1.2 키 관리

- **TD별 고유 키**: 각 TD는 독립적인 암호화 키 사용
- **하드웨어 키 생성**: CPU 내부에서 키 생성 및 관리
- **키 격리**: 다른 TD나 호스트에서 키 접근 불가

3.2 페이지 테이블 보호

3.2.1 Secure EPT (Extended Page Tables)

```
일반 VM의 메모리 매핑:
Guest Virtual Address → Guest Physical Address → Host Physical Address
↑
하이퍼바이저가 제어 가능


TD의 메모리 매핑:
Guest Virtual Address → Guest Physical Address → Secure Host Physical Address
↑
TDX 모듈만 제어 가능
```

3.2.2 메모리 접근 검증

```
// 의사 코드: TDX 메모리 접근 검증
```

```
fn access_td_memory(td_id: u32, address: u64, operation: MemOp) -> Result<()> {
// 1. TD 소유권 확인
if !is_td_memory(address, td_id) {
return Err(AccessDenied);
}

// 2. 암호화 키 확인
let key = get_td_key(td_id)?;

// 3. 메모리 암호화/복호화
match operation {
MemOp::Read => decrypt_and_read(address, key),
MemOp::Write => encrypt_and_write(address, key, data),
}
}
```

3.3 DMA 보호

3.3.1 IOMMU 통합

- **DMA 격리**: 디바이스가 TD 메모리에 직접 접근 차단
- **보안 DMA**: 승인된 디바이스만 제한적 접근 허용
- **메모리 태깅**: TD 메모리에 보안 태그 부여

---

4. 증명(Attestation) 시스템

4.1 증명의 목적

증명 시스템은 **"이 코드가 정말 TEE 환경에서 실행되고 있는가?"**를 외부에서 검증할 수 있게 해줍니다.

4.2 TDX Quote 구조

```
TDX Quote 구조:

┌──────────────────────────────────────────────────────────────────┐

│                        Quote Header                              │

│               - Version, Signature Type, Key ID                  │

├──────────────────────────────────────────────────────────────────┤

│                         TD Report                                │

│  ┌─────────────────┐  ┌─────────────────┐  ┌───────────────┐  │

│  │       MRTD      │  │    MRCONFIGID   │  │    MROWNER    │  │

│  │    (TD 측정값)   │  │  (설정 측정값)   │  │ (소유자 측정) │  │

│  └─────────────────┘  └─────────────────┘  └───────────────┘  │

│  ┌─────────────────┐  ┌─────────────────┐  ┌───────────────┐  │

│  │     RTMR0-3     │  │    Report Data  │  │   기타 필드   │  │

│  │   (런타임 측정)  │  │ (사용자 데이터)  │  │               │  │

│  └─────────────────┘  └─────────────────┘  └───────────────┘  │

├──────────────────────────────────────────────────────────────────┤

│                          Signature                               │
```

```
│              - Intel의 개인키로 서명된 디지털 서명              │
└──────────────────────────────────────────────────────────────────┘
```

4.3 증명 과정

4.3.1 Quote 생성 과정

```
1. 애플리케이션이 Report Data 준비
↓
2. TD 내부에서 TDCALL 호출
↓
3. TDX 모듈이 현재 TD 상태 수집
- MRTD (TD 측정값)
- RTMR (런타임 측정값)
- 기타 보안 속성
↓
4. Intel의 증명 키로 디지털 서명
↓
5. TDX Quote 생성 완료
```

4.3.2 Quote 검증 과정

```
1. 검증자가 TDX Quote 수신
↓
2. Intel 공개키로 서명 검증
```

```
↓
3. 측정값 검증
- 예상 MRTD와 비교
- RTMR 값 검증
- 설정값 확인
↓
4. Report Data 검증
- 애플리케이션별 검증 로직
↓
5. 증명 성공/실패 결정
```

4.4 실제 증명 코드 예시

4.4.1 Quote 요청 (TD 내부) - 실제 dstack 코드

```
// guest-agent/src/rpc_service.rs에서 실제 구현
async fn get_quote(self, request: RawQuoteArgs) -> Result<GetQuoteResponse> {
fn pad64(data: &[u8]) -> Option<[u8; 64]> {
if data.len() > 64 {
return None;
}
let mut padded = [0u8; 64];
padded[..data.len()].copy_from_slice(data);
Some(padded)
```

```
}

let  report_data = pad64(&request.report_data).context("Report data is too long"

if  self.state.config().simulator.enabled {
return  simulate_quote(
self.state.config(),
report_data,
&self.state.inner.vm_config,
);
}

// 실제 TDX Quote 요청
let (_, quote) = tdx_attest::get_quote(&report_data, None)
.context("Failed to get quote")?;

let  event_log = read_event_logs().context("Failed to decode event log")?;
let  event_log = serde_json::to_string(&event_log)
.context("Failed to serialize event log")?;

Ok(GetQuoteResponse {
quote,
event_log,
```

```
report_data: report_data.to_vec(),
vm_config: self.state.inner.vm_config.clone(),
})
}


// tdx-attest/src/linux.rs에서 실제 시스템 호출
pub fn get_quote(
report_data: &TdxReportData,
att_key_id_list: Option<&[TdxUuid]>,
) -> Result<(TdxUuid, Vec<u8>)> {
let mut att_key_id = TdxUuid([0; TDX_UUID_SIZE as usize]);
let mut quote_ptr = ptr::null_mut();
let mut quote_size = 0;


let error = unsafe {
let key_id_list_ptr = att_key_id_list
.map(|list| list.as_ptr() as *const tdx_uuid_t)
.unwrap_or(ptr::null());
tdx_att_get_quote(
report_data as *const TdxReportData as *const tdx_report_data_t,
key_id_list_ptr,
att_key_id_list.map_or(0, |list| list.len() as u32),
```

```
&mut att_key_id as *mut TdxUuid as *mut tdx_uuid_t,
&mut quote_ptr,
&mut quote_size,
0,
)
};

if error != _tdx_attest_error_t::TDX_ATTEST_SUCCESS {
return Err(error.into());
}

let quote = unsafe {
slice::from_raw_parts(quote_ptr, quote_size as usize).to_vec()
};

unsafe {
tdx_att_free_quote(quote_ptr);
}

Ok((att_key_id, quote))
}
```

4.4.2 Quote 검증 (외부 검증자)

```
use tdx_attest::QuoteVerifier;

fn verify_tdx_quote(quote: &[u8], expected_mrtd: &[u8]) -> Result<bool, Error>
let verifier = QuoteVerifier::new()?;

// 1. 서명 검증
verifier.verify_signature(quote)?;

// 2. Quote 파싱
let parsed_quote = verifier.parse_quote(quote)?;

// 3. 측정값 검증
if parsed_quote.td_report.mrtd != expected_mrtd {
return Err(Error::MeasurementMismatch);
}

// 4. 추가 검증 로직
verify_additional_attributes(&parsed_quote)?;

Ok(true)
}
```

5. 측정(Measurement)과 신뢰 체인

5.1 측정의 개념

측정(Measurement)은 **실행되는 코드와 데이터의 암호화 해시값**입니다. 이를 통해 시스템이 예상된 상태에서 실행되고 있는지 확인할 수 있습니다.

5.2 측정 레지스터

5.2.1 MRTD (Measurement Register of TD)

```
MRTD 계산 과정:

Initial MRTD = 0

↓

MRTD = SHA384(MRTD || TDVF_CODE)

↓

MRTD = SHA384(MRTD || KERNEL_CODE)

↓

MRTD = SHA384(MRTD || INITRD_DATA)

↓

Final MRTD (TD의 고유 식별자)
```

5.2.2 RTMR (Runtime Measurement Registers)

```
RTMR0: 부트로더 측정값

RTMR1: 운영체제 측정값

RTMR2: 애플리케이션 측정값
```

```
RTMR3: 사용자 정의 측정값 (dstack에서 사용)
```

5.3 신뢰 체인 구축

5.3.1 부팅 과정의 신뢰 체인

```
1. CPU 하드웨어 (신뢰 루트)
↓ 측정 및 검증
2. TDVF (TDX Virtual Firmware)
↓ 측정 및 검증
3. 부트로더 (GRUB 등)
↓ 측정 및 검증
4. 커널 (Linux)
↓ 측정 및 검증
5. 초기 램디스크 (initrd)
↓ 측정 및 검증
6. 사용자 애플리케이션
```

5.3.2 dstack의 실제 측정 과정

```
// tdx-attest/src/lib.rs에서 실제 RTMR3 확장 구현
pub fn extend_rtmr3(event: &str, payload: &[u8]) -> anyhow::Result<()> {
use anyhow::Context;
// This code is not defined in the TCG specification.
```

```
// See https://trustedcomputinggroup.org/wp-content/uploads/PC-ClientSpecific_Pl

let event_type = 0x08000001;

let index = 3;

let log = eventlog::TdxEventLog::new(index, event_type, event.to_string(), payl

extend_rtmr(index, event_type, log.digest).context("Failed to extend RTMR")?;

log_rtmr_event(&log).context("Failed to log RTMR event")

}




// tdx-attest/src/linux.rs에서 실제 RTMR 확장 시스템 호출

pub fn extend_rtmr(index: u32, event_type: u32, digest: [u8; 48]) -> Result<()

let event = tdx_rtmr_event_t {

version: 1,

rtmr_index: index as u64,

extend_data: digest,

event_type,

event_data_size: 0,

event_data: Default::default(),

};

let error = unsafe { tdx_att_extend(&event) };

if error != _tdx_attest_error_t::TDX_ATTEST_SUCCESS {

return Err(error.into());

}

Ok(())
```

```
}


// guest-agent/src/rpc_service.rs에서 이벤트 발생 시 호출
async fn emit_event(self, request: EmitEventArgs) -> Result<()> {
if self.state.config().simulator.enabled {
return Ok(());
}
tdx_attest::extend_rtmr3(&request.event, &request.payload)
}
```

---

6. 암호화 키 관리

6.1 키 계층 구조

```
키 계층 구조:

┌──────────────────────────────────────────────────────────────┐
│                      Hardware Root Key                       │
│                     (CPU 내부, 접근 불가)                     │
├──────────────────────────────────────────────────────────────┤
│                         Platform Key                         │
│                  (플랫폼별 고유 키, Intel 관리)               │
├──────────────────────────────────────────────────────────────┤
```

```
│                        TD Sealing Key                        │
│                    (TD별 고유 키, MRTD 기반)                  │
├──────────────────────────────────────────────────────────────┤
│                        Application Keys                      │
│                (애플리케이션별 파생 키, 용도별 분리)            │
└──────────────────────────────────────────────────────────────┘
```

6.2 키 유도 과정

6.2.1 TD Sealing Key 생성

```
// 의사 코드: TD Sealing Key 유도
fn derive_td_sealing_key(td_id: u32) -> Result<[u8; 32], Error> {
// 1. TD 정보 수집
let mrtd = get_mrtd(td_id)?;
let mrconfigid = get_mrconfigid(td_id)?;
let mrowner = get_mrowner(td_id)?;

// 2. 키 유도 재료 준비
let key_material = [mrtd, mrconfigid, mrowner].concat();

// 3. 하드웨어 키와 함께 HKDF 수행
let hardware_key = get_hardware_root_key()?;
let sealing_key = hkdf_expand(hardware_key, &key_material, 32)?;
```

```
Ok(sealing_key)
}
```

6.2.2 실제 애플리케이션 키 유도 구현

```
// ra-tls/src/kdf.rs에서 실제 키 유도 함수
/// Derives a key using HKDF-SHA256.
pub fn derive_ecdsa_key(
input_key_material: &[u8],
context_data: &[&[u8]],
key_size: usize,
) -> Result<Vec<u8>, Unspecified> {
let salt = Salt::new(HKDF_SHA256, b"RATLS");
let pseudo_rand_key: Prk = salt.extract(input_key_material);
let output_key_material: Okm<AnySizeKey> =
pseudo_rand_key.expand(context_data, AnySizeKey(key_size))?;
let mut result = vec![0u8; key_size];
output_key_material.fill(&mut result)?;
Ok(result)
}

/// Derives a key pair from a given private key bytes.
```

```
pub fn derive_ecdsa_key_pair_from_bytes(
sk_bytes: &[u8],
context_data: &[&[u8]],
) -> Result<KeyPair> {
let derived_sk_bytes =
derive_ecdsa_key(sk_bytes, context_data, 32)
.or(Err(anyhow!("failed to derive key")))?;
let derived_sk = p256::SecretKey::from_slice(&derived_sk_bytes)
.context("failed to decode derived secret key")?;
let derived_sk_der = derived_sk
.to_pkcs8_der()
.context("failed to encode derived secret key")?;
let der = PrivateKeyDer::try_from(derived_sk_der.as_bytes())
.map_err(|err| anyhow!("failed to decode derived secret key: {err}"))?;
let key = KeyPair::from_der_and_sign_algo(&der, &PKCS_ECDSA_P256_SHA256)
.context("failed to create derived key pair")?;
Ok(key)
}


// guest-agent/src/rpc_service.rs에서 실제 키 요청 처리
async fn get_key(self, request: GetKeyArgs) -> Result<GetKeyResponse> {
let k256_app_key = &self.state.inner.keys.k256_key;
```

```
// 경로 기반 키 유도
let derived_k256_key = derive_ecdsa_key(
k256_app_key,
&[request.path.as_bytes()],
32
).context("Failed to derive k256 key")?;

let derived_k256_key = SigningKey::from_slice(&derived_k256_key)
.context("Failed to parse k256 key")?;
let derived_k256_pubkey = derived_k256_key.verifying_key();

// 서명 체인 생성
let msg_to_sign = format!(
"{}:{}",
request.purpose,
hex::encode(derived_k256_pubkey.to_sec1_bytes())
);
let app_signing_key = SigningKey::from_slice(k256_app_key)
.context("Failed to parse app k256 key")?;
let digest = Keccak256::new_with_prefix(msg_to_sign);
let (signature, recid) = app_signing_key.sign_digest_recoverable(digest)?;
let mut signature = signature.to_vec();
signature.push(recid.to_byte());
```

```
Ok(GetKeyResponse {
key: derived_k256_key.to_bytes().to_vec(),
signature_chain: vec![signature, self.state.inner.keys.k256_signature.clone()],
})
}
```

6.3 키 봉인(Sealing)과 해제

6.3.1 데이터 봉인

```
fn seal_data(data: &[u8], policy: SealingPolicy) -> Result<Vec<u8>, Error> {
// 1. 현재 TD 상태 기반 키 유도
let sealing_key = derive_sealing_key_with_policy(policy)?;

// 2. 데이터 암호화
let nonce = generate_random_nonce();
let encrypted_data = aes_gcm_encrypt(data, &sealing_key, &nonce)?;

// 3. 봉인된 데이터 구조 생성
let sealed_data = SealedData {
policy,
nonce,
encrypted_data,
```

```
auth_tag: calculate_auth_tag(&encrypted_data, &sealing_key)?,
};

Ok(bincode::serialize(&sealed_data)?)
}
```

6.3.2 데이터 해제

```
fn unseal_data(sealed_data: &[u8]) -> Result<Vec<u8>, Error> {
// 1. 봉인된 데이터 파싱
let sealed: SealedData = bincode::deserialize(sealed_data)?;

// 2. 현재 상태로 키 재유도
let sealing_key = derive_sealing_key_with_policy(sealed.policy)?;

// 3. 인증 태그 검증
verify_auth_tag(&sealed.encrypted_data, &sealing_key, &sealed.auth_tag)?;

// 4. 데이터 복호화
let decrypted_data = aes_gcm_decrypt(
&sealed.encrypted_data,
&sealing_key,
&sealed.nonce
```

```
)?;


Ok(decrypted_data)

}
```

---

7. 보안 부팅 과정

7.1 TDX 부팅 시퀀스

```
부팅 시퀀스:

1. CPU 초기화

↓

2. SEAM 모듈 로딩

↓

3. TD 생성 요청 (하이퍼바이저)

↓

4. TDVF 로딩 및 측정

↓

5. 부트로더 로딩 및 측정

↓

6. 커널 로딩 및 측정

↓

7. initrd 로딩 및 측정
```

```
↓

8. 사용자 공간 초기화

↓

9. dstack-guest-agent 시작

↓

10. 컨테이너 실행
```

7.2 각 단계별 상세 과정

7.2.1 TDVF 단계

```
TDVF (TDX Virtual Firmware) 역할:

1. TD 초기 환경 설정

- 메모리 레이아웃 구성

- 보안 정책 설정

- 초기 측정값 생성



2. 부트로더 검증 및 로딩

- 부트로더 서명 검증

- 측정값 RTMR0에 기록

- 제어권 부트로더에 전달



3. 보안 서비스 초기화

- 난수 생성기 초기화
```

```
- 시간 동기화 설정
- 증명 서비스 준비
```

7.2.2 커널 부팅 단계

```
// 의사 코드: 커널 부팅 시 측정 과정
fn kernel_boot_measurement() -> Result<(), Error> {
// 1. 커널 이미지 측정
let kernel_hash = sha384(&kernel_image);
extend_rtmr(RTMR1, &kernel_hash)?;

// 2. 커널 명령행 측정
let cmdline_hash = sha384(kernel_cmdline.as_bytes());
extend_rtmr(RTMR1, &cmdline_hash)?;

// 3. initrd 측정
let initrd_hash = sha384(&initrd_image);
extend_rtmr(RTMR1, &initrd_hash)?;

// 4. 보안 설정 적용
apply_security_config()?;

Ok(())
```

```
}
```

7.3 dstack 특화 부팅 과정

7.3.1 게스트 에이전트 초기화

```
// dstack-guest-agent 초기화 과정
async fn initialize_guest_agent() -> Result<(), Error> {
// 1. TDX 환경 확인
verify_tdx_environment()?;

// 2. 하드웨어 보안 기능 초기화
initialize_security_features().await?;

// 3. KMS와 연결 설정
establish_kms_connection().await?;

// 4. 애플리케이션 키 준비
prepare_application_keys().await?;

// 5. Docker 환경 설정
setup_docker_environment().await?;

// 6. 웹 서비스 시작
```

```
start_web_services().await?;


Ok(())

}
```

7.3.2 보안 시간 동기화

```
// 보안 시간 동기화 (NTS 사용)

async fn secure_time_sync() -> Result<(), Error> {

// 1. 신뢰할 수 있는 NTS 서버 목록

let nts_servers = [

"time.cloudflare.com",

"nts.netnod.se",

"ptbtime1.ptb.de",

];


// 2. NTS 인증서 검증

for server in &nts_servers {

let cert = fetch_nts_certificate(server).await?;

verify_nts_certificate(&cert)?;

}


// 3. 시간 동기화 수행

let synchronized_time = perform_nts_sync(&nts_servers).await?;
```

```
// 4. 시스템 시간 설정
set_system_time(synchronized_time)?;

Ok(())
}
```

---

8. 런타임 보안 메커니즘

8.1 실행 중 보안 검증

8.1.1 지속적 무결성 검증

```
// 런타임 무결성 모니터링
struct IntegrityMonitor {
baseline_measurements: HashMap<String, [u8; 48]>,
monitoring_interval: Duration,
}

impl IntegrityMonitor {
async fn monitor_integrity(&self) -> Result<(), Error> {
loop {
// 1. 현재 시스템 상태 측정
```

```
let current_measurements = self.collect_measurements().await?;


// 2. 기준값과 비교
for (component, current_hash) in current_measurements {
if let Some(baseline) = self.baseline_measurements.get(&component) {
if current_hash != *baseline {
// 무결성 위반 감지
self.handle_integrity_violation(&component).await?;
}
}
}

// 3. 다음 검사까지 대기
tokio::time::sleep(self.monitoring_interval).await;
}
}
}
```

8.1.2 메모리 보호 검증

```
// 메모리 영역 보호 상태 검증
fn verify_memory_protection() -> Result<(), Error> {
// 1. TD 메모리 영역 확인
let td_memory_ranges = get_td_memory_ranges()?;
```

```
for range in td_memory_ranges {
// 2. 암호화 상태 확인
if !is_memory_encrypted(range.start, range.end)? {
return Err(Error::MemoryNotEncrypted);
}

// 3. 접근 권한 확인
if !verify_access_permissions(range)? {
return Err(Error::InvalidMemoryPermissions);
}
}

Ok(())
}
```

8.2 동적 측정 업데이트

8.2.1 실제 컨테이너 관리 구현

```
// guest-agent/src/guest_api_service.rs에서 실제 컨테이너 목록 조회
pub(crate) async fn list_containers() -> Result<ListContainersResponse> {
let docker = Docker::connect_with_defaults()
.context("Failed to connect to Docker")?;
```

```
let  containers = docker
.list_containers::<&str>(Some(ListContainersOptions {
all: true,
..Default::default()
}))
.await
.context("Failed to list containers")?;


Ok(ListContainersResponse {
containers: containers
.into_iter()
.map(|c| Container {
id: c.id.unwrap_or_default(),
names: c.names.unwrap_or_default(),
image: c.image.unwrap_or_default(),
state: c.state.unwrap_or_default(),
status: c.status.unwrap_or_default(),
created: c.created.unwrap_or_default(),
ports: c.ports.unwrap_or_default()
.into_iter()
.map(|p| format!("{}:{}", p.private_port, p.public_port.unwrap_or(0)))
.collect(),
})
```

```
.collect(),
})
}


// guest-agent/src/http_routes.rs에서 실제 로그 스트리밍
#[get("/logs/<container_name>?<since>&<until>&<follow>&<text>&<timestamps>&<bare
fn get_logs(
container_name: String,
since: Option<&str>,
until: Option<&str>,
follow: Option<bool>,
text: Option<bool>,
timestamps: Option<bool>,
bare: Option<bool>,
tail: Option<&str>,
ansi: Option<bool>,
) -> EventStream![String] {
EventStream! {
let config = docker_logs::LogConfig {
since,
until,
follow: follow.unwrap_or(false),
```

```
text: text.unwrap_or(false),

timestamps: timestamps.unwrap_or(false),

bare: bare.unwrap_or(false),

tail,

ansi: ansi.unwrap_or(false),

};



let mut stream = match docker_logs::get_logs(&container_name, config) {

Ok(stream) => stream,

Err(e) => {

yield format!("data: {{\"error\": \"{}\"}}\n\n", e);

return;

}

};



while let Some(result) = stream.next().await {

match result {

Ok(log_line) => {

yield format!("data: {}\n\n", log_line);

}

Err(e) => {

yield format!("data: {{\"error\": \"{}\"}}\n\n", e);

break;
```

```
}
}
}
}
}

// docker_logs 모듈에서 실제 Docker API 호출
pub fn get_logs(
container_name: &str,
config: LogConfig,
) -> Result<impl Stream<Item = Result<String, bollard::errors::Error>>> {
let LogConfig {
since,
until,
follow,
text,
timestamps,
bare,
tail,
ansi,
} = config;

let docker = Docker::connect_with_local_defaults()?;
```

```
let options = LogsOptions {
stdout: true,
stderr: true,
since,
until,
timestamps,
follow,
tail: tail.unwrap_or("all").to_string(),
};

Ok(docker
.logs(container_name, Some(options))
.map(move |result| result.map(|m| log_to_json(m, text, bare, ansi))))
}
```

8.2.2 설정 변경 시 측정

```
// 시스템 설정 변경 시 측정 업데이트
fn update_system_config(new_config: &SystemConfig) -> Result<(), Error> {
// 1. 현재 설정과 비교
let current_config = load_current_config()?;
if current_config == *new_config {
```

```
    return  Ok(()); // 변경 없음
    }

    // 2. 새 설정 측정
    let  config_measurement = sha384(&bincode::serialize(new_config)?);

    // 3. RTMR3에 기록
    extend_rtmr3(&config_measurement)?;

    // 4. 설정 적용
    apply_system_config(new_config)?;

    // 5. 변경 로그 기록
    log::info!("System configuration updated, new measurement: {}",
    hex::encode(&config_measurement));

    Ok(())
    }
```

---

9. 실제 동작 시나리오

9.1 웹 애플리케이션 배포 시나리오

9.1.1 전체 과정 개요

```
사용자 요청: "nginx 웹서버를 TEE에서 실행하고 싶다"
↓
1. docker-compose.yaml 작성
2. dstack-vmm에 업로드
3. CVM 생성 및 부팅
4. 컨테이너 실행
5. 네트워크 설정
6. 외부 접근 가능
```

9.1.2 상세 동작 과정

1단계: CVM 생성

```rust
// VMM에서 CVM 생성 과정
async fn create_cvm_for_app(app_config: &AppConfig) -> Result<CvmId, Error> {
// 1. 리소스 할당
let resources = allocate_resources(app_config.memory, app_config.cpu)?;

// 2. TDX 모듈에 TD 생성 요청
let td_params = TdParams {
memory_size: resources.memory,
cpu_count: resources.cpu,
initial_measurement: calculate_initial_measurement(app_config)?,
```

```
};

let td_id = tdx_module.create_td(td_params).await?;

// 3. 네트워크 설정
let network_config = setup_cvm_network(td_id).await?;

// 4. CVM 정보 저장
let cvm_id = store_cvm_info(td_id, resources, network_config).await?;

Ok(cvm_id)
}
```

2단계: 보안 부팅

```
// CVM 내부에서의 부팅 과정
async fn secure_boot_process() -> Result<(), Error> {
// 1. TDX 환경 검증
verify_tdx_environment()?;

// 2. 하드웨어 보안 기능 초기화
initialize_hardware_security().await?;
```

```
// 3. 보안 시간 동기화 (옵션)
if config.secure_time_enabled {
secure_time_sync().await?;
}

// 4. KMS 연결 및 인증
let kms_client = connect_to_kms().await?;
authenticate_with_kms(&kms_client).await?;

// 5. 애플리케이션 키 요청
let app_keys = request_application_keys(&kms_client).await?;

// 6. Docker 환경 준비
setup_docker_environment(&app_keys).await?;

Ok(())
}
```

3단계: 실제 애플리케이션 상태 관리

```
// guest-agent/src/models.rs에서 대시보드 모델
#[derive(Template)]
#[template(path = "dashboard.html")]
pub struct Dashboard {
```

```
pub app_name: String,
pub app_id: String,
pub instance_id: String,
pub device_id: String,
pub key_provider_info: String,
pub tcb_info: String,
pub containers: Vec<Container>, // 실제 컨테이너 목록
pub system_info: SystemInfo,
pub public_sysinfo: bool,
}


// guest-agent/src/http_routes.rs에서 실제 대시보드 렌더링
#[get("/")]
async fn dashboard(state: &State<AppState>) -> Result<Template, Status> {
let handler = GuestApiHandler::new(state.clone());
let app_info = get_info(state, false).await.map_err(|_| Status::InternalServerE


let system_info = handler.sys_info().await.unwrap_or_default();


// 실제 실행 중인 컨테이너 목록 조회
let containers = list_containers().await.unwrap_or_default().containers;
```

```
let model = crate::models::Dashboard {
app_name: app_info.app_name,
app_id: app_info.app_id,
instance_id: app_info.instance_id,
device_id: hex::encode(&app_info.device_id),
key_provider_info: app_info.key_provider_info,
tcb_info: app_info.tcb_info,
containers, // Docker에서 조회한 실제 컨테이너들
system_info,
public_sysinfo: state.config().app_compose.public_sysinfo,
};

Ok(Template::render("dashboard", &model))
}

// guest-api/src/lib.rs에서 실제 컨테이너 구조체
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct Container {
pub id: String,
pub names: Vec<String>,
pub image: String,
pub state: String,
pub status: String,
```

```
pub created: i64,
pub ports: Vec<String>,
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct ListContainersResponse {
pub containers: Vec<Container>,
}
```

4단계: 네트워크 노출

```
// Gateway를 통한 외부 접근 설정
async fn setup_external_access(cvm_id: &CvmId, ports: &[u16]) -> Result<(), Er
// 1. CVM의 WireGuard IP 획득
let cvm_ip = get_cvm_wireguard_ip(cvm_id).await?;

// 2. Gateway에 라우팅 규칙 등록
for port in ports {
let route_rule = RouteRule {
domain_pattern: format!("{}-{}.app.example.com", cvm_id, port),
target_ip: cvm_ip,
target_port: *port,
tls_termination: true,
```

```
};

gateway_client.add_route_rule(route_rule).await?;
}

// 3. TLS 인증서 요청 (필요시)
if config.auto_tls {
request_tls_certificate(cvm_id).await?;
}

Ok(())
}
```

9.2 데이터베이스 애플리케이션 시나리오

9.2.1 민감한 데이터 처리

```
// 데이터베이스 암호화 키 관리
async fn setup_database_encryption() -> Result<DatabaseKeys, Error> {
// 1. 애플리케이션별 키 유도
let master_key = derive_app_key("database", "master_encryption").await?;
let backup_key = derive_app_key("database", "backup_encryption").await?;
let log_key = derive_app_key("database", "log_encryption").await?;
```

```
// 2. 키 순환 설정
let key_rotation_schedule = setup_key_rotation(Duration::from_secs(86400 * 30))


// 3. 데이터베이스 설정에 키 적용
let db_config = DatabaseConfig {
encryption_key: master_key,
backup_encryption_key: backup_key,
log_encryption_key: log_key,
key_rotation_schedule,
};


Ok(DatabaseKeys::from(db_config))
}
```

9.2.2 백업 데이터 봉인

```
// 데이터베이스 백업 시 데이터 봉인
async fn create_sealed_backup(database_data: &[u8]) -> Result<Vec<u8>, Error>
// 1. 백업 정책 정의
let sealing_policy = SealingPolicy {
require_same_mrtd: true, // 동일한 TD에서만 해제 가능
require_same_mrowner: true, // 동일한 소유자만 해제 가능
require_same_app: true, // 동일한 애플리케이션만 해제 가능
};
```

```
// 2. 데이터 압축
let compressed_data = compress_data(database_data)?;

// 3. 데이터 봉인
let sealed_backup = seal_data(&compressed_data, sealing_policy).await?;

// 4. 백업 메타데이터 추가
let backup_metadata = BackupMetadata {
timestamp: SystemTime::now(),
database_version: get_database_version()?,
backup_size: sealed_backup.len(),
checksum: sha256(&sealed_backup),
};

// 5. 최종 백업 패키지 생성
let final_backup = create_backup_package(sealed_backup, backup_metadata)?;

Ok(final_backup)
}
```

---

10. 보안 위협과 대응

10.1 주요 보안 위협

10.1.1 하드웨어 공격

- **물리적 접근**: 서버에 대한 물리적 접근을 통한 공격
- **사이드 채널**: 전력 분석, 타이밍 공격 등
- **펌웨어 변조**: BIOS/UEFI 펌웨어 변조

10.1.2 소프트웨어 공격

- **하이퍼바이저 공격**: 악의적인 하이퍼바이저를 통한 공격
- **호스트 OS 공격**: 호스트 운영체제 침해
- **네트워크 공격**: 네트워크를 통한 데이터 탈취

10.2 TDX의 보안 대응

10.2.1 하드웨어 기반 보호

```
// 하드웨어 보안 기능 검증
fn verify_hardware_security() -> Result<(), Error> {
// 1. TDX 지원 확인
if !cpu_supports_tdx() {
return Err(Error::TdxNotSupported);
}

// 2. 메모리 암호화 확인
if !is_memory_encryption_enabled() {
return Err(Error::MemoryEncryptionDisabled);
```

```
}

// 3. 보안 부팅 확인
if !is_secure_boot_enabled() {
return Err(Error::SecureBootDisabled);
}

// 4. 증명 기능 확인
if !is_attestation_available() {
return Err(Error::AttestationNotAvailable);
}

Ok(())
}
```

10.2.2 런타임 보안 모니터링

```
// 실시간 보안 모니터링
struct SecurityMonitor {
threat_detector: ThreatDetector,
incident_handler: IncidentHandler,
}
```

```
impl SecurityMonitor {
async fn monitor_security_events(&self) -> Result<(), Error> {
let mut event_stream = self.threat_detector.get_event_stream().await?;


while let Some(event) = event_stream.next().await {
match event {
SecurityEvent::MemoryViolation(details) => {
self.handle_memory_violation(details).await?;
}
SecurityEvent::UnauthorizedAccess(details) => {
self.handle_unauthorized_access(details).await?;
}
SecurityEvent::IntegrityViolation(details) => {
self.handle_integrity_violation(details).await?;
}
SecurityEvent::NetworkAnomaly(details) => {
self.handle_network_anomaly(details).await?;
}
}
}


Ok(())
}
```

```
async fn handle_memory_violation(&self, details: MemoryViolationDetails) -> Re

// 1. 위반 내용 분석

let threat_level = self.analyze_threat_level(&details);



// 2. 즉시 대응

match threat_level {

ThreatLevel::Critical => {

// 시스템 즉시 격리

self.isolate_system().await?;

// 관리자 알림

self.notify_administrators(&details).await?;

}

ThreatLevel::High => {

// 해당 프로세스 종료

self.terminate_suspicious_process(details.process_id).await?;

}

ThreatLevel::Medium => {

// 추가 모니터링 강화

self.enhance_monitoring(details.process_id).await?;

}

ThreatLevel::Low => {

// 로그 기록만
```

```
log::warn!("Low-level memory violation detected: {:?}", details);

}

}


Ok(())

}

}
```

10.3 dstack의 추가 보안 기능

10.3.1 Certificate Transparency 모니터링

```
// CT 로그 모니터링을 통한 인증서 위조 탐지

struct CertificateMonitor {

known_public_keys: HashSet<Vec<u8>>,

ct_log_clients: Vec<CtLogClient>,

}


impl CertificateMonitor {

async fn monitor_certificate_transparency(&self) -> Result<(), Error> {

for ct_client in &self.ct_log_clients {

let recent_certificates = ct_client.get_recent_certificates().await?;


for cert in recent_certificates {
```

```
// 1. 도메인 확인
if cert.matches_domain("*.app.example.com") {
// 2. 공개키 확인
let public_key = cert.get_public_key()?;


if !self.known_public_keys.contains(&public_key) {
// 3. 알 수 없는 인증서 발견
self.handle_unauthorized_certificate(cert).await?;
}
}
}
}

Ok(())
}

async fn handle_unauthorized_certificate(&self, cert: Certificate) -> Result<(
// 1. 즉시 알림
let alert = SecurityAlert {
severity: AlertSeverity::Critical,
message: format!("Unauthorized certificate detected for domain: {}", cert.domain
certificate_details: cert.clone(),
timestamp: SystemTime::now(),
```

```
};

self.send_security_alert(alert).await?;

// 2. 자동 대응
// CAA 레코드 재확인 및 업데이트
self.update_caa_records().await?;

// 3. 인증서 폐기 요청
self.request_certificate_revocation(&cert).await?;

Ok(())
}
}
```

10.3.2 네트워크 트래픽 분석

```
// 네트워크 트래픽 이상 탐지
struct  NetworkAnalyzer {
baseline_patterns: TrafficPatterns,
anomaly_detector: AnomalyDetector,
}
```

```rust
impl NetworkAnalyzer {
async fn analyze_network_traffic(&self) -> Result<(), Error> {
let mut traffic_stream = self.get_traffic_stream().await?;

while let Some(packet) = traffic_stream.next().await {
// 1. 트래픽 패턴 분석
let pattern = self.extract_pattern(&packet)?;

// 2. 이상 탐지
if self.anomaly_detector.is_anomalous(&pattern, &self.baseline_patterns) {
self.handle_network_anomaly(packet, pattern).await?;
}

// 3. 패턴 업데이트
self.update_baseline_patterns(pattern).await?;
}

Ok(())
}
}
```

결론

TEE(Trusted Execution Environment)는 하드웨어 기반의 강력한 보안 격리 환경을 제공합니다. Intel TDX를 통해 구현된 dstack은 다음과 같은 핵심 보안 메커니즘을 제공합니다:

핵심 보안 원리

1. **하드웨어 기반 격리**: CPU 레벨에서의 메모리 및 실행 격리
2. **암호화 기반 보호**: 모든 데이터의 자동 암호화
3. **측정 기반 신뢰**: 실행 코드의 암호화 해시를 통한 무결성 검증
4. **증명 시스템**: 외부에서 검증 가능한 실행 환경 증명

실제 동작 과정

1. **보안 부팅**: 신뢰 체인을 통한 안전한 시스템 초기화
2. **키 관리**: 하드웨어 기반 키 유도 및 관리
3. **런타임 보호**: 실행 중 지속적인 보안 모니터링
4. **네트워크 보안**: 암호화된 통신 및 자동 TLS 관리

이러한 메커니즘들이 결합되어 클라우드 환경에서도 온프레미스 수준의 보안을 제공하며, 개발자는 기존의 Docker 워크플로우를 그대로 사용하면서도 최고 수준의 보안을 얻을 수 있습니다.